Task 1

1) X={(0,0),(0,1),(1,0),(1,1)} Y={0,1}
labelled examples are ((0,0),0),((0,1),1),((1,0),1),((1,1),0)

2)Class 0 are (0,0) and (1,1)
Class 1 are (0,1) and (1,0)

3)The points belonging to each class lie on opposite diagonal corners. Therefore, no single straight line can separate both Class 0 points from both Class 1 points.

4)A single affine transformation followed by a sigmoid will fail to perfectly learn XOR. Training may reduce the loss somewhat, but the model cannot achieve perfect classification because its decision boundary is linear.

Task2

Design:A feed-forward neural network with 2 input units, 2 hidden units using a nonlinear activation such as sigmoid or tanh, and 1 sigmoid output unit. Binary cross-entropy is used as the loss function and the parameters are optimised using gradient-based optimisation.

Validation:The trained model is evaluated on all four XOR input patterns. Successful learning is indicated by decreasing BCE loss and correct classification of all four examples using a 0.5 decision threshold, ideally achieving 100% accuracy.

1)The hidden nonlinearity is necessary because XOR cannot be separated by a single straight line. If the hidden layer used only linear operations, stacking layers would still behave like one linear model. The nonlinear activation lets the network create a more complex, nonlinear decision boundary and therefore learn the XOR relationship.

2)Sigmoid is appropriate because the network needs to produce one probability-like value representing a binary yes/no prediction. Binary cross-entropy is designed for comparing such probability predictions with binary targets and gives a strong penalty when the model is confidently wrong.

Reflection Questions

1. What did the XOR experiment demonstrate about the difference between depth and nonlinearity?

ANS-: The XOR experiment showed that depth alone is not sufficient ,stacking affine/linear layers without nonlinear activations is still equivalent to a single linear transformation. The hidden nonlinearity is what allows the network to represent the nonlinear XOR decision boundary.

2. In your successful run, what evidence showed that backpropagation supplied a useful learning signal rather than merely a nonzero gradient?

ANS:-he useful evidence was that, after repeated gradient-based updates, the loss decreased and the model correctly classified all four XOR examples,

3. Why did identical/zero weight initialisation prevent the two hidden units from learning distinct features?

ANS:-Identically initialized hidden units produce identical outputs for every input and therefore receive identical gradients. Their parameters remain identical after every update, so the two units cannot specialize and learn distinct features, this is the symmetry problem.

4. How did changing the hidden activation affect the gradient you observed? Distinguish the scientific explanation from the engineering observation.

ANS:- **Scientific explanation:** different activations have different derivatives, so they propagate gradients differently through the network for example, sigmoid can have smaller gradients in saturated regions, while tanh and ReLU have different gradient behaviour.

Engineering observation: in this particular four-point experiment, the measured first-layer gradient norms and final losses differed between sigmoid, tanh, and ReLU. This does not prove that one activation is universally best it only describes their behaviour under this architecture, initialization, and training setup.

5. Why must the output layer and loss be selected together according to the task?

ANS:-The output representation must match the task and the loss. For binary classification, one logit with BCEWithLogitsLoss is appropriate; for three mutually exclusive classes, three logits with CrossEntropyLoss are appropriate .

6. Give one example where the LLM improved your engineering productivity and one example where human verification was essential.

ANS:-The LLM improved engineering productivity by quickly generating and modifying the PyTorch code for the XOR, symmetry, activation, and multiclass experiments. Human verification was essential to run the code, inspect the actual numerical outputs, and confirm that the results and interpretation were correct, rather than assuming generated code or predicted results were correct.

7. Which tests in this laboratory would you keep if the model were scaled up, and which would become too expensive (for example, exhaustive finite-difference gradient checks)?

ANS:-I would keep loss monitoring, validation accuracy, prediction checks on representative data, gradient checks, reproducibility tests, and small-scale sanity tests. Exhaustive checks such as finite-difference gradients for every parameter or testing every possible input would become too computationally expensive, so they would instead be performed on a small subset of parameters or a small test model.